

Лабораторная работа № 7.

OpenSSL, собственный CA и HTTPS

Тема: Создание собственного Root CA, выпуск TLS-сертификата и настройка HTTPS на Nginx

Цель: Создать учебный CA, выпустить TLS-сертификат для web01, настроить Nginx HTTPS и проверить доверие клиента к сертификату.

Дисциплина: Основные сервисы Linux для предприятия

Лабораторная работа № 7. OpenSSL, собственный CA и HTTPS

Цель лабораторной работы — создать собственный учебный удостоверяющий центр (CA), выпустить с его помощью TLS-сертификат для веб-сервера Nginx, настроить HTTPS и проверить, что клиент доверяет выпущенному сертификату.

Исходный сценарий лекции предполагает три отдельные роли: ca01.company.test — удостоверяющий центр, web01.company.test — веб-сервер, client01 — клиент. В основной лабораторной работе используются две виртуальные машины: роли ca01 и web01 объединяются на одной VM server01, а client01 остаётся отдельной VM. Это сделано только для упрощения учебного стенда.

Важно: в реальной инфраструктуре CA желательно отделять от прикладных серверов. Особенно строго должен защищаться закрытый ключ CA. Объединение ролей CA и веб-сервера в этой работе является учебным допущением, а не рекомендацией для рабочей сети.

Основная схема лабораторного стенда

VM1: server01
IP: 192.168.10.20/24
Роли:
- Root CA
- OpenSSL
- Nginx
- HTTPS-сервис web01.company.test

VM2: client01
IP: 192.168.10.30/24
Роли:
- клиент HTTPS
- доверенное хранилище CA
- curl / openssl
- браузер при наличии GUI

VirtualBox Internal Network: tls-lab

client01 ————— HTTPS :443 —————> server01

- Root CA
- web01.key
- web01.crt
- Nginx

1. Что должно получиться в конце

- На server01 создан собственный Root CA: закрытый ключ ca.key и публичный сертификат ca.crt.
- Для веб-сервера создан отдельный закрытый ключ web01.key.
- Создан запрос на подпись web01.csr.
- Root CA подписал CSR и выпустил серверный сертификат web01.crt.
- Сертификат web01.crt содержит SAN с DNS-именем web01.company.test и IP-адресом 192.168.10.20.
- Nginx использует web01.crt и web01.key и слушает TCP-порт 443.
- На client01 установлен ca.crt в доверенное хранилище.
- Команда curl https://web01.company.test работает без параметра --cacert.
- openssl s_client показывает Verify return code: 0 (ok).
- При наличии графической среды браузер открывает https://web01.company.test без предупреждения о недоверенном издателе.

2. Исходные параметры

Параметры лаборатории

Параметр	Значение
Сеть VirtualBox	Internal Network: tls-lab
server01	192.168.10.20/24
client01	192.168.10.30/24
Имя HTTPS-сервиса	web01.company.test
HTTPS-порт	443/tcp
Имя Root CA	Company Test Root CA
SAN	DNS:web01.company.test, IP:192.168.10.20
Срок Root CA	3650 дней
Срок web01.crt	365 дней

3. Подготовка сети VirtualBox

3.1. Настройка server01

На server01 удобно использовать два сетевых адаптера. Первый нужен только для доступа в Интернет и установки пакетов. Второй используется для изолированной лабораторной сети.

1. Adapter 1: NAT.
2. Adapter 2: Internal Network.
3. Имя Internal Network: tls-lab.
4. На внутреннем интерфейсе назначить адрес 192.168.10.20/24.
5. Gateway на внутреннем интерфейсе не задавать.

После загрузки server01 определите имена интерфейсов. В VirtualBox они часто называются enp0s3, enp0s8, но полагаться на это заранее нельзя.

```
Проверка сетевых интерфейсов на server01
```

```
ip -br a
```

В выводе найдите интерфейс NAT и интерфейс сети tls-lab. На внутреннем интерфейсе должен быть адрес 192.168.10.20/24.

```
Проверка маршрутов на server01
```

```
ip route
```

Default route должен идти через NAT-интерфейс. Для сети 192.168.10.0/24 должен быть отдельный локальный маршрут через внутренний интерфейс.

3.2. Настройка client01

На client01 основной адаптер подключите к той же Internal Network tls-lab. Назначьте адрес 192.168.10.30/24. Постоянный доступ в Интернет клиенту для этой лабораторной не требуется.

Если на client01 ещё не установлены curl, openssl или ca-certificates, можно временно включить второй адаптер NAT, установить пакеты, а затем отключить NAT.

```
Проверка client01
```

```
ip -br a  
ip route
```

На внутреннем интерфейсе client01 должен быть адрес 192.168.10.30/24.

3.3. Проверка связи между ВМ

До настройки сертификатов сначала убедитесь, что обычная IP-связность работает.

```
Выполнить на client01
```

```
ping -c 4 192.168.10.20
```

Если ответы приходят, client01 видит server01. Если ping не работает, сначала исправьте сеть VirtualBox и IP-адресацию. TLS бессмысленно диагностировать до исправления обычной сетевой связи.

4. Настройка имени web01.company.test

Отдельный DNS-сервер в этой лабораторной не разворачивается. Чтобы client01 мог обращаться к серверу по имени web01.company.test, добавим локальную запись в /etc/hosts.

```
Открыть hosts на client01
```

```
sudo nano /etc/hosts
```

Добавьте в конец файла строку:

```
Запись hosts
```

```
192.168.10.20    web01.company.test web01
```

Сохраните файл и проверьте разрешение имени.

Проверка имени

```
getent hosts web01.company.test
ping -c 2 web01.company.test
```

Ожидаемый результат: имя web01.company.test разрешается в адрес 192.168.10.20. Это важно, потому что именно это DNS-имя будет включено в сертификат.

5. Установка необходимых программ на server01

На server01 понадобятся OpenSSL для работы с ключами и сертификатами, Nginx для HTTPS-сайта, ca-certificates для системного хранилища сертификатов и OpenSSH Server для удобного копирования ca.crt на client01.

Установка пакетов

```
sudo apt update
sudo apt install openssl nginx ca-certificates openssh-server -y
```

После установки включите Nginx и SSH.

Запуск служб

```
sudo systemctl enable --now nginx
sudo systemctl enable --now ssh
```

Проверьте, что программы доступны.

Проверка

```
openssl version
sudo nginx -v
systemctl status nginx
```

Для Nginx ожидается состояние active (running).

6. Создание собственного Root CA

На этом этапе server01 временно играет роль ca01. Мы создаём собственный корневой удостоверяющий центр, который сможет подписывать сертификаты внутренних сервисов.

6.1. Создание структуры каталогов CA

Создадим отдельную структуру внутри /root/ca. Каталог private будет содержать закрытый ключ CA, certs — сертификаты, csr — запросы на подпись.

Создание каталогов

```
sudo mkdir -p /root/ca/private /root/ca/certs /root/ca/csr
```

Ограничим доступ к каталогу CA.

Права на каталоги

```
sudo chmod 700 /root/ca /root/ca/private
```

Теперь обычные пользователи не смогут просматривать закрытые материалы CA.

6.2. Создание закрытого ключа CA — ca.key

Закрытый ключ CA — самый чувствительный файл в лаборатории. С его помощью CA будет подписывать сертификаты серверов. Если этот ключ попадёт к постороннему, он сможет выпускать сертификаты от имени нашего CA.

Создание ca.key

```
sudo openssl genrsa -aes256 \
-out /root/ca/private/ca.key 4096
```

OpenSSL попросит придумать парольную фразу для ca.key. Введите её дважды и запомните. При вводе символы на экране не отображаются — это нормально.

Параметр -aes256 защищает закрытый ключ шифрованием на диске. Число 4096 задаёт размер RSA-ключа.

После создания ограничьте права доступа.

Защита ca.key

```
sudo chmod 400 /root/ca/private/ca.key
```

Проверьте наличие файла.

Проверка

```
sudo ls -l /root/ca/private/ca.key
```

Файл ca.key нельзя копировать на client01, помещать в общую папку или прикладывать к отчёту.

6.3. Создание корневого сертификата CA — ca.crt

Закрытый ключ ca.key уже есть, но сам по себе он ещё не является сертификатом. Теперь на его основе создадим публичный корневой сертификат ca.crt.

Именно ca.crt впоследствии устанавливается клиентам как доверенный сертификат. В отличие от ca.key, файл ca.crt не является секретом.

Создание ca.crt

```
sudo openssl req -x509 -new \
-key /root/ca/private/ca.key \
-sha256 \
-days 3650 \
-out /root/ca/certs/ca.crt \
-subj "/CN=Company Test Root CA"
```

После запуска OpenSSL запросит парольную фразу от ca.key. Введите тот пароль, который задали на предыдущем этапе.

Параметр -x509 означает, что создаётся сразу X.509-сертификат, а не обычный CSR. Параметр -days 3650 задаёт срок действия примерно 10 лет. Параметр -subj задаёт имя нашего CA без интерактивного опроса.

Проверьте, что файл создан.

Проверка наличия ca.crt

```
sudo ls -l /root/ca/certs/ca.crt
```

Теперь выведите основные сведения сертификата.

Проверка полей Root CA

```
sudo openssl x509 \
-in /root/ca/certs/ca.crt \
-noout -subject -issuer -dates
```

В выводе Subject и Issuer должны быть одинаковыми: CN = Company Test Root CA. Это нормально для Root CA: его корневой сертификат самоподписанный.

После этого этапа у нас есть два разных файла: ca.key — секретный закрытый ключ CA; ca.crt — публичный корневой сертификат, который можно передавать клиентам.

Что получилось после этапа 6

```
/root/ca/
├── private/
│   └── ca.key    ← секретный закрытый ключ CA
├── certs/
│   └── ca.crt    ← публичный сертификат CA
└── csr/          ← здесь позже будет web01.csr
```

7. Создание закрытого ключа веб-сервера web01

Теперь server01 начинает выполнять вторую роль — web01. Веб-сервер должен иметь собственный закрытый ключ. Нельзя использовать ca.key как ключ Nginx.

Создание рабочего каталога web01

```
sudo mkdir -p /root/web01-tls
```

Создание web01.key

```
sudo openssl genrsa \
-out /root/web01-tls/web01.key 2048
```

Этот ключ создаём без парольной фразы, чтобы Nginx мог автоматически запускаться после перезагрузки без ручного ввода пароля.

Права на web01.key

```
sudo chmod 600 /root/web01-tls/web01.key
```

Проверьте файл.

Проверка

```
sudo ls -l /root/web01-tls/web01.key
```

web01.key остаётся только на server01. На client01 этот файл не копируется.

8. Создание CSR для web01

CSR — Certificate Signing Request, запрос на подпись сертификата. Он содержит открытый ключ web01 и сведения о владельце, но не содержит закрытый ключ web01.key.

Создание web01.csr

```
sudo openssl req -new \  
-key /root/web01-tls/web01.key \  
-out /root/ca/csr/web01.csr \  
-subj "/CN=web01.company.test"
```

Команда использует web01.key, извлекает из него открытый ключ и создаёт запрос для CA.

Проверьте CSR.

Проверка CSR

```
sudo openssl req \  
-in /root/ca/csr/web01.csr \  
-noout -subject
```

Ожидается Subject с CN = web01.company.test.

9. Добавление SAN

Современные клиенты проверяют имя сервера по расширению Subject Alternative Name (SAN). Одного Common Name недостаточно.

Создадим файл расширений для будущего сертификата web01.

Создание файла расширений

```
sudo nano /root/ca/csr/web01.ext
```

Вставьте:

Содержимое web01.ext

```
basicConstraints=CA:FALSE
```

```
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:web01.company.test,IP:192.168.10.20
```

Здесь CA:FALSE означает, что сертификат web01 не является сертификатом удостоверяющего центра. extendedKeyUsage=serverAuth указывает назначение — серверная TLS-аутентификация.

SAN содержит DNS:web01.company.test и IP:192.168.10.20. Поэтому клиент сможет корректно проверять сертификат при обращении по имени web01.company.test и, при необходимости, по этому IP.

10. Подписание CSR и выпуск web01.crt

Теперь Root CA подпишет запрос web01.csr своим закрытым ключом ca.key. Результатом станет серверный сертификат web01.crt.

Выпуск сертификата

```
sudo openssl x509 -req \
  -in /root/ca/csr/web01.csr \
  -CA /root/ca/certs/ca.crt \
  -CAkey /root/ca/private/ca.key \
  -CAcreateserial \
  -out /root/web01-tls/web01.crt \
  -days 365 \
  -sha256 \
  -extfile /root/ca/csr/web01.ext
```

OpenSSL попросит парольную фразу от ca.key. Введите пароль CA.

Параметр -CA указывает публичный сертификат CA, а -CAkey — закрытый ключ CA. Параметр -extfile добавляет SAN и другие расширения из web01.ext.

10.1. Проверка подписи

Убедимся, что web01.crt действительно подписан нашим CA.

Проверка цепочки

```
sudo openssl verify \
  -CAfile /root/ca/certs/ca.crt \
  /root/web01-tls/web01.crt
```

Ожидаемый результат:

Успешная проверка

```
/root/web01-tls/web01.crt: OK
```

10.2. Проверка полей и SAN

Проверка сертификата web01

```
sudo openssl x509 \
  -in /root/web01-tls/web01.crt \
```

```
-noout -subject -issuer -dates -ext subjectAltName
```

Проверьте три вещи: Subject содержит web01.company.test; Issuer содержит Company Test Root CA; SAN содержит DNS:web01.company.test и IP Address:192.168.10.20.

11. Установка сертификата и ключа для Nginx

Nginx должен знать, где находятся серверный сертификат и соответствующий закрытый ключ.

Копирование сертификата

```
sudo cp /root/web01-tls/web01.crt \  
/etc/ssl/certs/web01.crt
```

Копирование закрытого ключа

```
sudo cp /root/web01-tls/web01.key \  
/etc/ssl/private/web01.key
```

Настроим права. Сертификат можно читать всем, а закрытый ключ должен быть доступен только root.

Права

```
sudo chown root:root /etc/ssl/private/web01.key  
sudo chmod 600 /etc/ssl/private/web01.key  
sudo chown root:root /etc/ssl/certs/web01.crt  
sudo chmod 644 /etc/ssl/certs/web01.crt
```

Проверка

```
sudo ls -l /etc/ssl/private/web01.key  
ls -l /etc/ssl/certs/web01.crt
```

12. Создание тестовой веб-страницы

Создадим отдельный каталог сайта и простую HTML-страницу, чтобы было видно, что HTTPS действительно работает.

Создание каталога

```
sudo mkdir -p /var/www/web01
```

Создание index.html

```
sudo tee /var/www/web01/index.html > /dev/null <<'EOF'  
<!doctype html>  
<html lang="ru">  
<head>
```

```
<meta charset="utf-8">
<title>HTTPS работает</title>
</head>
<body>
  <h1>HTTPS работает</h1>
  <p>web01.company.test защищён сертификатом Company Test Root CA.</p>
</body>
</html>
EOF
```

Проверьте созданный файл.

Проверка страницы

```
cat /var/www/web01/index.html
```

13. Настройка Nginx для HTTPS

Создадим отдельный virtual host для web01.company.test.

Открыть конфигурацию

```
sudo nano /etc/nginx/sites-available/web01
```

Вставьте:

Конфигурация web01

```
server {
    listen 443 ssl;
    server_name web01.company.test;

    root /var/www/web01;
    index index.html;

    ssl_certificate      /etc/ssl/certs/web01.crt;
    ssl_certificate_key  /etc/ssl/private/web01.key;

    ssl_protocols TLSv1.2 TLSv1.3;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

listen 443 ssl включает HTTPS. server_name должен совпадать с именем в SAN. ssl_certificate указывает сертификат web01.crt, а ssl_certificate_key — соответствующий закрытый ключ web01.key.

Включите сайт.

Создание ссылки

```
sudo ln -s /etc/nginx/sites-available/web01 \
    /etc/nginx/sites-enabled/web01
```

Если команда сообщает, что файл уже существует, это нормально — ссылка уже была создана.

До перезагрузки Nginx обязательно проверьте синтаксис.

Проверка конфигурации

```
sudo nginx -t
```

Ожидается:

Успешный nginx -t

```
syntax is ok
test is successful
```

Только после успешной проверки примените конфигурацию.

Применение

```
sudo systemctl reload nginx
```

Проверьте, слушает ли Nginx порт 443.

Проверка HTTPS-порта

```
sudo ss -tulpen | grep ':443'
```

Если строка LISTEN для :443 отсутствует, клиент не сможет подключиться по HTTPS.

14. Локальная проверка HTTPS на server01

До переноса CA на клиент убедимся, что сам Nginx отдаёт правильный сертификат.

Проверка TLS

```
sudo openssl s_client \
  -connect 127.0.0.1:443 \
  -servername web01.company.test \
  -CAfile /root/ca/certs/ca.crt \
  -verify_return_error < /dev/null
```

Параметр -servername передаёт имя web01.company.test как SNI. Параметр -CAfile говорит OpenSSL доверять нашему ca.crt.

В конце вывода найдите:

Ожидаемый результат

```
Verify return code: 0 (ok)
```

Если код проверки не равен 0, не переходите к client01 — сначала исправьте сертификат или конфигурацию Nginx.

15. Подготовка client01

На client01 понадобятся curl, OpenSSL, ca-certificates и SSH-клиент.

Установка пакетов

```
sudo apt update
sudo apt install curl openssl ca-certificates openssh-client -y
```

Если client01 не имеет Интернета, временно включите NAT для установки пакетов, затем снова отключите.

16. Передача ca.crt с server01 на client01

Клиенту нужен только публичный сертификат CA — ca.crt. Закрытые ключи ca.key и web01.key клиенту не передаются.

16.1. Подготовка публичной копии на server01

Файл /root/ca/certs/ca.crt лежит в каталоге root, поэтому обычный пользователь не сможет забрать его через scp. Создадим публичную копию в домашнем каталоге пользователя.

На server01

```
sudo cp /root/ca/certs/ca.crt /home/user/company-test-root-ca.crt
sudo chown user:user /home/user/company-test-root-ca.crt
sudo chmod 644 /home/user/company-test-root-ca.crt
```

Если ваш локальный пользователь на server01 называется не user, замените user и /home/user на фактическое имя и путь.

Проверка на server01

```
ls -l /home/user/company-test-root-ca.crt
```

16.2. Копирование с client01

На client01 выполните:

Копирование CA

```
scp user@192.168.10.20:/home/user/company-test-root-ca.crt /tmp/
```

При первом SSH-подключении может появиться вопрос о доверии к ключу хоста. Введите yes, затем пароль локального пользователя user на server01.

После успешного копирования проверьте файл.

Проверка

```
ls -l /tmp/company-test-root-ca.crt
```

Важно: здесь используется пароль Linux-пользователя server01, а не пароль CA.

17. Демонстрация ошибки доверия до установки CA

Сначала специально проверим HTTPS до добавления ca.crt в системное доверенное хранилище client01. Это показывает, зачем вообще нужен доверенный CA.

Обычный curl до импорта CA

```
curl https://web01.company.test
```

Ожидается ошибка проверки сертификата: client01 пока не знает Company Test Root CA и поэтому не может подтвердить доверие к web01.crt.

Теперь явно укажем CA только для одной команды.

Проверка с явным CA

```
curl --cacert /tmp/company-test-root-ca.crt \  
https://web01.company.test
```

Если HTML-страница отображается, значит Nginx, web01.crt и сеть настроены правильно. Проблема была только в отсутствии доверия к CA.

18. Импорт Root CA в доверенное хранилище client01

Добавим Company Test Root CA в системное доверенное хранилище Linux.

Копирование CA

```
sudo cp /tmp/company-test-root-ca.crt \  
/usr/local/share/ca-certificates/company-test-root-ca.crt
```

Файлы в /usr/local/share/ca-certificates с расширением .crt учитываются утилитой update-ca-certificates.

Обновление хранилища

```
sudo update-ca-certificates
```

В выводе ожидается добавление нового сертификата, например 1 added.

Теперь система доверяет сертификатам, подписанным Company Test Root CA.

19. Финальная проверка на client01

19.1. Проверка curl без --cacert

HTTPS-запрос

```
curl https://web01.company.test
```

Теперь команда должна вывести HTML-страницу без ошибки сертификата. Это ключевой результат лабораторной.

19.2. Проверка через openssl s_client

Проверка TLS-цепочки

```
openssl s_client \  
-connect web01.company.test:443 \  
-servername web01.company.test \  
-CAfile /etc/ssl/certs/ca-certificates.crt \  
-verify_return_error < /dev/null
```

В конце вывода должно быть:

Ожидаемый результат

```
Verify return code: 0 (ok)
```

19.3. Проверка в браузере

Если client01 имеет графическую среду, откройте браузер и перейдите по адресу:

URL

```
https://web01.company.test
```

Страница должна открыться по HTTPS без предупреждения о неизвестном издателе. Если браузер использует собственное хранилище сертификатов, может потребоваться отдельный импорт CA в настройки браузера.

20. Что произошло в результате

В лаборатории была построена минимальная инфраструктура доверия PKI.

Логика доверия

```
Company Test Root CA  
|  
| ca.key подписывает  
|  
v  
web01.crt  
|  
| Nginx показывает сертификат клиенту  
|  
v  
web01.company.test  
|  
| HTTPS  
|  
v  
client01  
|  
| доверяет ca.crt
```

client01 не знает и не получает закрытый ключ CA. Он получает только публичный ca.crt. Этого достаточно, чтобы проверить подпись web01.crt.

Если ca.crt установлен в доверенное хранилище и имя web01.company.test присутствует в SAN, клиент принимает серверный сертификат как доверенный.

21. Типовые ошибки и диагностика

Диагностика HTTPS

Симптом	Вероятная причина	Что проверить
Connection refused	Nginx не слушает 443	systemctl status nginx; ss -tulpen grep :443
web01.company.test не найден	Нет записи в hosts	getent hosts web01.company.test
unknown issuer / unable to get local issuer	CA не установлен	update-ca-certificates; наличие ca.crt
hostname mismatch	Имя отсутствует в SAN	openssl x509 -ext subjectAltName
certificate verify failed	Неверный CA или цепочка	openssl verify; openssl s_client
Permission denied в Nginx	Неверные права на web01.key	ls -l /etc/ssl/private/web01.key
Key values mismatch	Сертификат не соответствует ключу	сравнить публичные ключи
certificate expired	Истёк срок сертификата	openssl x509 -dates
certificate is not yet valid	Неверное системное время	date; timedatectl

21.1. Проверка соответствия сертификата и закрытого ключа

Если есть подозрение, что web01.crt выпущен не для текущего web01.key, сравните их публичные ключи.

Хэш публичного ключа из сертификата

```
openssl x509 -in /etc/ssl/certs/web01.crt \
  -pubkey -noout | sha256sum
```

Хэш публичного ключа из private key

```
sudo openssl pkey -in /etc/ssl/private/web01.key \
  -pubout | sha256sum
```

Хэши должны совпасть.

22. Что приложить к отчёту

6. Схему стенда с двумя VM и пояснением, что server01 совмещает роли ca01 и web01.
7. IP-адреса server01 и client01.
8. Вывод openssl x509 для ca.crt с Subject, Issuer и сроком действия.
9. Вывод openssl x509 для web01.crt с SAN.
10. Результат openssl verify с ОК.

11. Фрагмент конфигурации Nginx.
12. Результат `sudo nginx -t`.
13. Вывод `ss`, подтверждающий прослушивание TCP 443.
14. Ошибка `curl` до импорта CA.
15. Успешный `curl` после импорта CA.
16. `Verify return code: 0 (ok)`.
17. Скриншот браузера с `https://web01.company.test`, если используется GUI.

23. Запасной сценарий: одна Debian-VM + Windows-хост

Если второй Linux-клиент использовать неудобно, роль client01 может выполнять основной компьютер Windows. Тогда нужна только одна Debian-VM: она совмещает CA и Nginx/web01.

Этот вариант сохраняет учебную цель: сертификат создаётся на Debian, а доверие проверяется на отдельной клиентской системе — Windows.

Схема запасного сценария

```
Windows 11 – клиент
- hosts
- браузер
- curl.exe
- доверенное хранилище CA
    |
    | HTTPS
    v
Debian VM server01
- Root CA
- OpenSSL
- Nginx
- web01.company.test
```

23.1. Выбор сетевого режима

Для одной VM удобны два варианта: Host-only Adapter или Bridged Adapter.

Host-only предпочтительнее для учебного стенда: Windows и VM видят друг друга, но VM не выставляется в общую сеть. Bridge можно использовать, если нужен доступ к VM как к обычному узлу локальной сети.

Если используется Bridge, IP VM может отличаться от 192.168.10.20. Узнайте фактический адрес:

На Debian VM

```
ip -br a
```

Например, VM получила 192.168.1.50. Тогда этот адрес нужно использовать в Windows hosts и включить в SAN сертификата.

23.2. Настройка Windows hosts

Откройте Блокнот от имени администратора и откройте файл:

Windows hosts

```
C:\Windows\System32\drivers\etc\hosts
```

Добавьте строку с фактическим IP Debian-VM.

Пример

```
192.168.1.50    web01.company.test
```

После этого очистите DNS-кэш Windows:

PowerShell

```
ipconfig /flushdn
```

Проверьте из PowerShell:

PowerShell

```
ping web01.company.test
```

Имя должно разрешаться в IP Debian-VM.

PS: Можно не менять C:\Windows\System32\drivers\etc\hosts, тогда проверку нужно будет проводить не по имени сервера, а по его IP.

23.3. SAN для запасного сценария

Если IP VM отличается от 192.168.10.20, файл web01.ext нужно изменить до выпуска сертификата.

Пример для IP 192.168.1.50

```
basicConstraints=CA:FALSE
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:web01.company.test,IP:192.168.1.50
```

После изменения SAN снова выполните выпуск web01.crt. Сертификат должен соответствовать тому имени или IP, по которому Windows обращается к серверу.

23.4. Подготовка ca.crt для Windows

Windows-клиенту нужен только публичный корневой сертификат ca.crt. Закрытые ключи ca.key и web01.key на Windows не копируются.

На Debian создайте копию ca.crt в домашнем каталоге обычного пользователя:

На Debian

```
sudo cp /root/ca/certs/ca.crt /home/user/company-test-root-ca.crt
sudo chown user:user /home/user/company-test-root-ca.crt
sudo chmod 644 /home/user/company-test-root-ca.crt
```

Если пользователь называется иначе, замените user на фактическое имя.

23.5. Копирование ca.crt на Windows

Откройте PowerShell на Windows и перейдите, например, в Downloads.

```
PowerShell  
cd $HOME\Downloads
```

Скопируйте сертификат через SCP. В примере IP Debian-VM — 192.168.1.50.

```
PowerShell  
scp user@192.168.1.50:/home/user/company-test-root-ca.crt .
```

При первом подключении подтвердите ключ SSH словом yes, затем введите пароль Linux-пользователя user.

После копирования проверьте:

```
PowerShell  
dir company-test-root-ca.crt
```

На Windows должен находиться только публичный company-test-root-ca.crt.

23.6. Импорт Root CA в Windows

Дважды щёлкните по файлу company-test-root-ca.crt в Проводнике. Откроется окно сертификата.

Проверьте, что сертификат выдан Company Test Root CA и кем выдан также Company Test Root CA. Совпадение нормально: это самоподписанный корневой сертификат.

Нажмите «Установить сертификат...».

Выберите расположение «Локальный компьютер». Для этого потребуются права администратора.

Выберите «Поместить все сертификаты в следующее хранилище», нажмите «Обзор» и укажите «Доверенные корневые центры сертификации» (Trusted Root Certification Authorities).

Нажмите «Далее» → «Готово». Если Windows предупреждает об установке нового корневого сертификата, подтвердите действие для этого учебного CA.

Смысл этого действия: Windows начинает доверять не отдельному сайту, а удостоверяющему центру Company Test Root CA. Поэтому сертификаты, подписанные этим CA, могут проходить проверку доверия.

23.7. Проверка CA в Windows

Нажмите Win+R и запустите:

```
Команда  
certlm.msc
```

Откройте «Доверенные корневые центры сертификации» → «Сертификаты». В списке должен быть Company Test Root CA.

23.8. Финальная проверка с Windows

В PowerShell выполните:

```
PowerShell

curl.exe https://web01.company.test
```

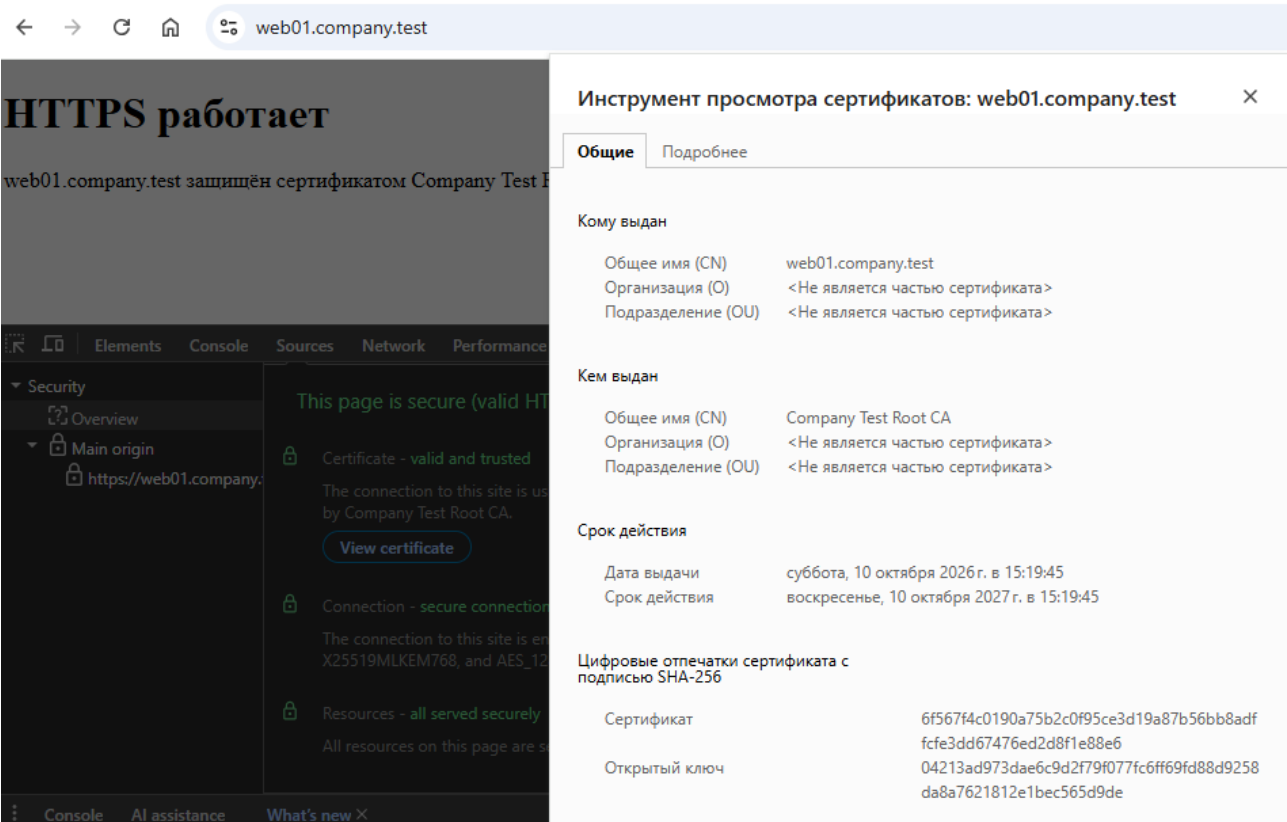
При успешной настройке команда должна получить HTML-страницу без ошибки сертификата.

Затем откройте в браузере:

```
URL

https://web01.company.test
```

Ожидаемый результат: страница открывается по HTTPS без предупреждения о неизвестном издателе.



Если ошибка связана с именем сертификата, проверьте, что DNS:web01.company.test находится в SAN. Если ошибка связана с неизвестным издателем, проверьте установку Company Test Root CA в Trusted Root Certification Authorities.

24. Сравнение основного и запасного сценариев

Сценарии лабораторной

Элемент	Основной: 2 VM	Запасной: 1 VM + Windows
---------	----------------	--------------------------

Элемент	Основной: 2 ВМ	Запасной: 1 ВМ + Windows
CA	server01	Debian VM
Nginx/web01	server01	Debian VM
Клиент	client01 Linux	Windows-хост
Сеть	Internal Network tls-lab	Host-only или Bridge
Разрешение имени	/etc/hosts	Windows hosts
Импорт CA	update-ca-certificates	Trusted Root Certification Authorities
Проверка	curl + openssl + браузер	curl.exe + браузер

25. Контрольные вопросы

18. Что такое CA и какую задачу он решает?
19. Почему ca.key является секретным, а ca.crt можно передавать клиентам?
20. Почему web01 должен иметь собственный закрытый ключ web01.key?
21. Что такое CSR и почему он не содержит закрытого ключа?
22. Какую роль играет SAN в серверном сертификате?
23. Почему одного Common Name недостаточно?
24. Почему до установки ca.crt client01 не доверяет web01.crt?
25. Что означает Verify return code: 0 (ok)?
26. Почему Nginx должен слушать порт 443?
27. Что произойдёт, если server_name не совпадает с SAN?
28. Почему в реальной инфраструктуре CA лучше отделять от веб-сервера?
29. Почему Host-only обычно безопаснее Bridge для учебной лаборатории?